Application Q&A; & Technical Project Summary

Project: Adaptive Self-Verifying Lung Cancer AI | **Live App:** https://112005.streamlit.app/

---

1. Vercel Deployment Troubleshooting & Streamlit Hosting

Q: I'm deploying this project on Vercel, but it shows an error: *'Error: Found app.py but it does not export a top-level app or application'*.

Root Cause: Vercel expects Python files in the root (like app.py) to be stateless WSGI/ASGI serverless endpoints exporting an app object (e.g. Flask or FastAPI).

Streamlit applications (streamlit run app.py) require a stateful background Python server process and WebSockets, which Vercel Serverless Functions do not support. Additionally, heavy ML dependencies like PyTorch (torch) exceed Vercel's 250 MB function limit.

Resolution & Deployment: Successfully deployed the Streamlit app to **Streamlit Community Cloud**.

Live Production URL: https://112005.streamlit.app/ (Verified active with 3D CT slice viewers, Grad-CAM XAI, Self-Verification engine, and MC uncertainty calibration).

2. Project Highlight (2-Minute Link Walkthrough)

Q: If we only have two minutes with that link, what should we look at, and why? Point us at the specific file, feature, or decision you are proudest of.

*Check out the **Self-Verification & Clinical Diagnosis tab** (src/self_verification.py), which cross-checks predicted malignancy risk against radiological nodule features and Monte Carlo uncertainty to automatically flag potential false positives. It turns a black-box 3D neural network into an adaptive, self-auditing decision support system built for high-stakes clinical safety.*

3. Confidently Wrong LLM Output & Code Guardrails

Q: Describe a time an LLM gave you a confidently wrong answer inside something you were building. How did you catch it, and what did you change so it would not happen again?

What it got wrong: While writing code to process 3D medical CT volumes using pydicom and SimpleITK, an LLM generated plausible-looking code that assumed array dimensions were structured as (C, H, W, D) instead of standard spatial (D, H, W). It also hallucinated a non-existent parameter name for Hounsfield Unit windowing.

How I caught it: My unit tests failed immediately during tensor shape assertions (pytest), and visual slice overlays showed rotated, corrupted organ geometry during manual inspection.

What I put in place afterwards:
1. Created strict automated unit tests and pytest shape/range assertions for all data ingestion pipelines.
2. Added Pydantic schema validation on data structures so unexpected shapes or hallucinated API parameters throw runtime errors immediately instead of failing silently downstream.

4. 300-Page Annual Financial Report Audit (1-Hour Challenge)

Q: You are handed a 300-page annual report and an AI system that has extracted the financials from it. You have one hour to decide whether a banker can put their name on that output. What do you check, and in what order?

First, I'd run automated math checks on the extracted data (Assets = Liabilities + Equity, subtotals matching line items). If basic arithmetic fails, the AI pipeline is unreliable and I can stop right there without wasting the remaining 55 minutes.

If the math holds, I'd spot-check the 10 biggest numbers (Revenue, Net Income, Debt) directly against the raw PDF pages, double-checking units (millions vs. thousands) and multi-column alignment where LLMs often trip up. Finally, I'd check low-confidence flags and footnote restatements before signing off.

5. Self-Taught Endeavor in the Last 6 Months (Ungraded)

Q: What is something you taught yourself in the last six months that nobody assigned to you? Does not have to be technical.

Option A — Non-Technical (Fitness & Posture Mobility):
• *What it was:* Progressive calisthenics and shoulder/spine mobility training.
• *What set me off:* I noticed how stiff my posture and back were getting after sitting at a desk for long coding sessions, and I wanted to improve physical health without needing a gym membership.
• *How far I got:* Built a daily 20-minute mobility routine, fixed posture, and worked up from basic bodyweight exercises to holding a controlled 10-second freestanding handstand.

Option B — Technical (AI Calibration & Uncertainty Quantification):
• *What it was:* Post-hoc confidence calibration (Temperature Scaling) and Monte Carlo Dropout to measure epistemic and aleatoric uncertainty.
• *What set me off:* Noticed how black-box neural networks often output 99% confidence even when predicting on noisy or ambiguous data.
• *How far I got:* Implemented temperature scaling from scratch and built a pipeline that separates model uncertainty from data noise into a live clinical UI.

6. Strong Opinion / Open Closing Thoughts

Q: Anything we did not ask that we should know? A constraint, an unusual background, a question for us, a strong opinion about something.

Strong opinion on AI engineering: *High accuracy is only half the battle; the real engineering challenge is building safety systems around the model. Most real-world AI failures aren't caused by low accuracy, but by models being confidently wrong without any uncertainty bounds or verification rails.*